TINYORBIT · DESIGN DOCUMENT

evolve: a self-evolving layer over an agent harness

The model stays frozen. Everything the harness *says* to it (the system prompt, standing memory, lessons, skills and tool descriptions) becomes a versioned artifact called a **genome**. Two loops improve that artifact from feedback. An inner loop distills lessons after every run. An outer loop runs an archive-based evolutionary search over whole genomes, scored on a real benchmark and accepted only when the gain holds up on tasks it never trained on.

In one paragraph. `python -m tinyorbit.evolve run` starts from today's harness text (generation 0), asks an LLM mutator to read the failures of a parent genome and propose small SEARCH/REPLACE edits, filters the child through free guards and a staged evaluation cascade (smoke set, then train set), scores it as *train score minus a size penalty*, files it in a MAP-Elites archive split into islands, and promotes it to **champion** only if it also beats the current champion on a held-out split. Every attempt, kept or rejected, is written to a ledger that the next mutation reads. Runs are checkpointed and resumable. The champion is a plain directory you pass to any run with `--genome`.

Contents

0 The simple version, and what it costs
1 Why a self-evolving layer
2 The genome
3 Inner loop: reflection
4 Outer loop: the evolutionary search
5 The evaluator contract
6 SWE-bench as the evaluator
7 End to end: a run, step by step
8 Design choices and where they come from
9 Cost, noise and honest limits
10 What comes next
A CLI and file reference

tinyorbit, branch `feat/self-evolve` · `src/tinyorbit/evolve/` (~1,300 lines) and `bench/` (evaluator, scoring, splits) · 158 tests, no network · October 2026

0 · The simple version, and what it costs

Forget the machinery for a moment. Evolve does this:

1. **Run today's tinyorbit (generation 0)** on 10 hard problems. Say it solves 7/10. That is the score to beat.
2. **Change the genome a little.** An LLM reads why the 3 hard problems failed and edits the prompt, a skill or a lesson.
3. **Test the changed version.** First on 3 easy problems: if it now fails one, the change broke something, so throw it away (cheap). Then on the 10 hard problems: did it beat 7/10?
4. **If it did, check it on 10 *other* hard problems it has never seen.** It becomes the new best only if it also does better there. That proves the change helps in general, not just on the 10 it practised on.
5. **Repeat** steps 2 to 4. Each repeat is one **iteration**, and each one builds on the versions that did well.

The 3 easy problems are not what we improve: tinyorbit already solves them all. They are a safety check that a change did not break the agent. The score that counts is on the **hard** problems, and a change only wins if it also helps on hard problems it never trained on.

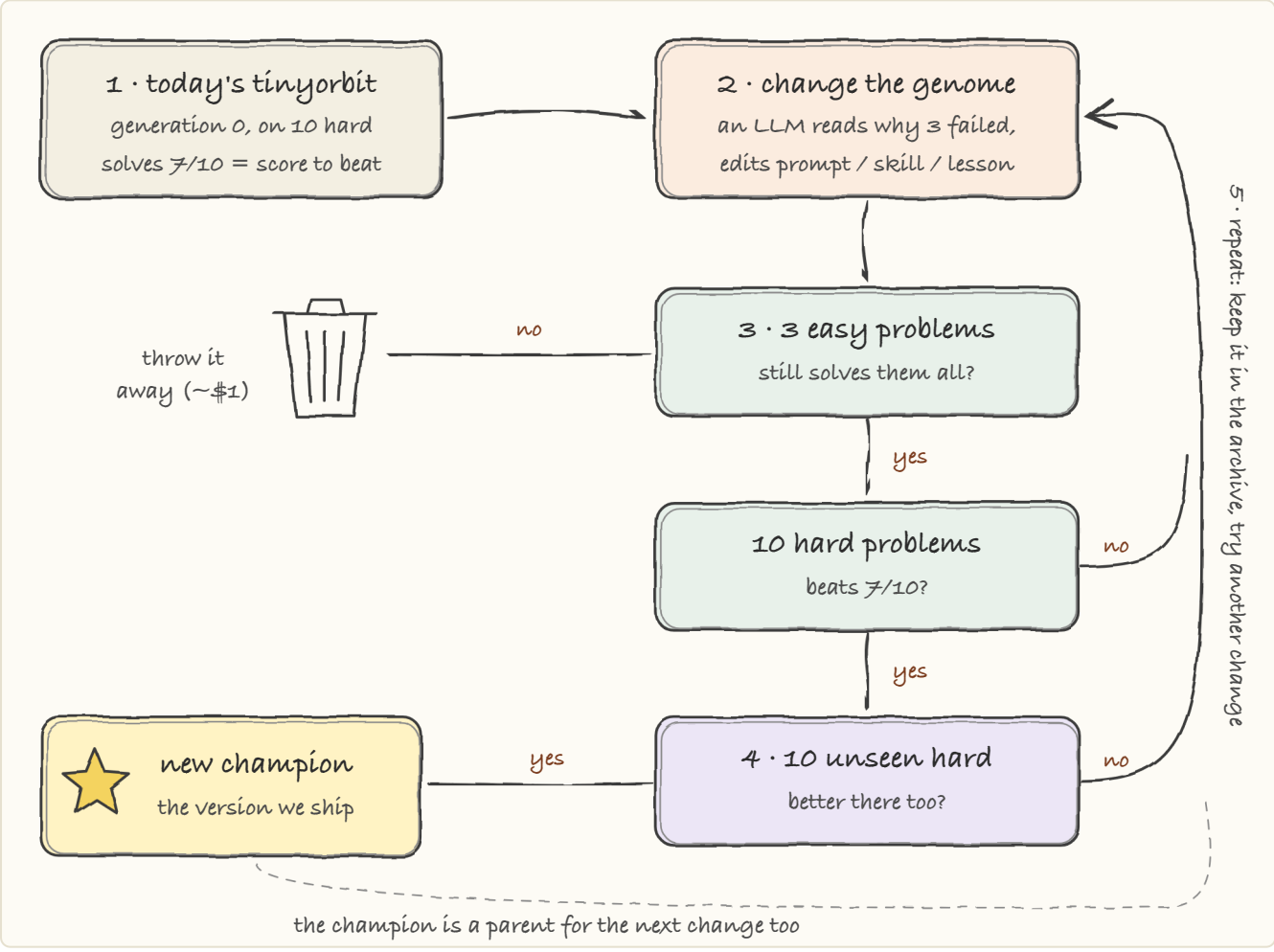


Figure 0. One iteration, in pencil. Each trip around the loop costs between about $1 (fails the easy check) and about $21 (goes all the way to the unseen problems).

What it costs

Measured prices from the earlier benchmark runs (claude-opus-5, 60-turn cap): **a hard problem costs about $1** per agent run, an easy one about $0.35. Scoring runs locally in Docker and costs nothing. The

LLM that proposes each change costs cents.

Step	Agent runs	Cost
Generation 0: 3 easy + 10 hard + 10 unseen hard (paid once)	23	≈ $21
An iteration thrown away at the easy check	3	≈ $1
An iteration that runs the 10 hard but does not beat the best	13	≈ $11
An iteration that also runs the 10 unseen hard	23	≈ $21
A change the guards reject before running anything (duplicate, broken edit)	0	$0

> **Total for a first run of 5 iterations: about $75** in a typical mix, with a floor of about $26 (every change fails the easy check) and a ceiling of about $130 (every change goes all the way). Add about $10 to re-run the final champion on the unseen problems once, to confirm it was not luck. Wall time is about 3 to 5 hours with two tasks in parallel.
>
> **For 10 iterations: about $135 typical**, between $31 and $235.

These are estimates, not caps. An evolved prompt that makes the agent work longer raises the per-problem cost, and a hard problem can run to the 60-turn limit. A run can be stopped at any moment and resumed later, losing nothing that has finished.

1 · Why a self-evolving layer

An agent is a harness plus a model. tinyorbit already owns the harness: the loop, the tools, the permission chain, the system prompt. When the agent fails a task, the failure usually lives in that harness text, not in the weights. The prompt never told it to reproduce the bug before fixing it. A tool description invites a lazy use. No lesson says "your own new test passing is not proof." Those are cheap to change and expensive to find by hand.

The self-evolving agent stack is usually drawn in layers, ordered from cheapest and safest to most powerful and riskiest:

Layer	What changes	In tinyorbit
Skills and memory	Text the agent reads: lessons, playbooks, standing instructions	**Built.** `lessons.md`, `memory.md`, `skills/`
Harness	The system prompt and tool descriptions	**Built.** `system_prompt.md`, `tools/*.md`
Harness code	The loop and tools themselves	Not yet. Sandboxing needed first
Weights	Fine-tuning on the agent's own trajectories	Out of scope: the model is frozen
Environment and orchestration	Curricula, task generation, multi-agent topologies	Out of scope for now

evolve starts at the narrowest surface that can fix a real failure: **editable text around a frozen model**. Every variant is Markdown. It can be read, diffed, reviewed, reverted and versioned in git. Nothing evolved is executable, so a bad generation can lower a score but cannot break the machine.

> **The central risk is that the loop fools itself.** A system that edits its own instructions against a 10-task benchmark will find edits that fit those 10 tasks. Three mechanisms exist to stop that, and they recur throughout this document. First, a **held-out gate**: a change is promoted only if it also improves a disjoint task split whose failures the mutator never sees. Second, a **size penalty**: fitness is charged for growth, so the cheapest way to fit the train set (appending task-specific hints) is taxed. Third, a **ledger**: every rejected idea is remembered and shown to the next proposer.

2 · The genome

A genome is a small directory of Markdown that the harness reads *on top of* its built-in defaults. It is the single artifact both loops evolve.

File	Role at run time
`system_prompt.md`	Replaces the static system prompt template. Must keep the `{cwd}` and `{platform}` placeholders. They are filled by plain string replacement, so stray braces in evolved text can never crash formatting.
`memory.md`	Harness-level standing instructions, loaded like a global `AGENTS.md`.
`lessons.md`	Lessons distilled from past runs (written by the inner loop), loaded as memory right after `memory.md`.
`skills/<name>/SKILL.md` `skills/<name>.md`	Extra on-demand playbooks, same format as project skills. A project skill with the same name wins.
`tools/<ToolName>.md`	Replaces the description the model sees for that tool. Unknown tool names are ignored.

How a run picks it up

`main.py --genome DIR` or `TINYORBIT_GENOME=DIR`. In `bootstrap.setup()` the overlay is loaded once and applied in a fixed order:

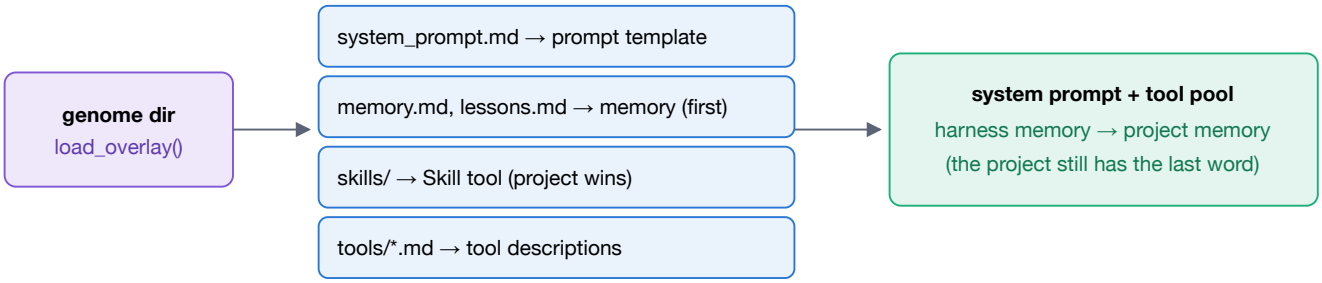


Figure 1. The overlay. Genome memory is injected *before* project memory, so a repository's own `AGENTS.md` still overrides evolved harness rules.

The overlay is not trust-gated the way project files are, because the operator passes it explicitly. It cannot escape its own directory: `Genome.save()` and every edit go through a **path whitelist** (a regex over exactly the five layouts above). An evolved edit to `../main.py` or `tools/../../x` is rejected before it is written.

Generation 0 is today's harness, byte for byte

`python -m tinyorbit.evolve seed DIR` exports the built-in prompt template and every tool's description as a genome. A test asserts that running with this seed produces the *identical* system prompt and tool schemas as running with no genome. This matters more than it looks. If the seed differed even slightly from the shipped harness, every evolved score would be measured against a baseline that never existed.

A genome has a **fingerprint** (the first 16 hex digits of a SHA-256 over its sorted files) and a **size** (its total characters). The fingerprint makes evaluation memoisable and deduplication free. The size feeds the penalty and the archive's grid.

---

3 · Inner loop: reflection

The inner loop is the cheap, per-run step: one extra model call after a session ends. It turns a finished trajectory into at most three general lessons in `lessons.md`, which the next run loads as memory. No weights change. This is online self-improvement through context.



Figure 2. One reflection. The transcript is condensed to keep the task statement and the ending (where the mistakes usually are), with "[N steps omitted]" in between.

Two guards keep the lessons file from rotting:

- **Verified outcomes beat self-reports.** The reflector is told what actually happened. Live, in `--print --genome DIR --reflect` mode, the only signal is the agent's own exit reason, labelled "*self-reported, unverified*". Offline, `python -m tinyorbit.evolve reflect --transcript T --outcome failed --feedback "FAIL_TO_PASS still red"` passes the ground truth. That is worth far more, because the most common failure is an agent sincerely believing it succeeded.
- **A bounded, deduplicated book.** Lessons are keyed by their normalized title. A new lesson with a known title replaces the old one and moves to the end. Past 20 lessons the oldest fall off. Known lessons are shown to the reflector so it does not restate them, and malformed items are dropped. The file format is a plain Markdown list with provenance in an HTML comment:

```
# Lessons learned

Distilled from past runs of this agent. Apply a lesson when its situation matches.

- **Reproduce before fixing**: Write a failing test from the issue text first; a new test
that passes proves nothing. <!-- outcome=failed source=astropy__astropy-13398 -->
```

Because lessons live in the genome, the outer loop can also rewrite, merge or delete them like any other artifact. Reflection adds; evolution curates.

4 · Outer loop: the evolutionary search

The outer loop searches over whole genomes. It is a clean-room implementation of the archive-plus-islands pattern popularized by program-evolution systems: an LLM proposes diffs, an evaluator scores them, and an archive keeps a diverse population rather than a single best. It lives in `loop.py` (the driver), `mutate.py` (the proposer), `edits.py` (the diff format) and `archive.py` (the population).

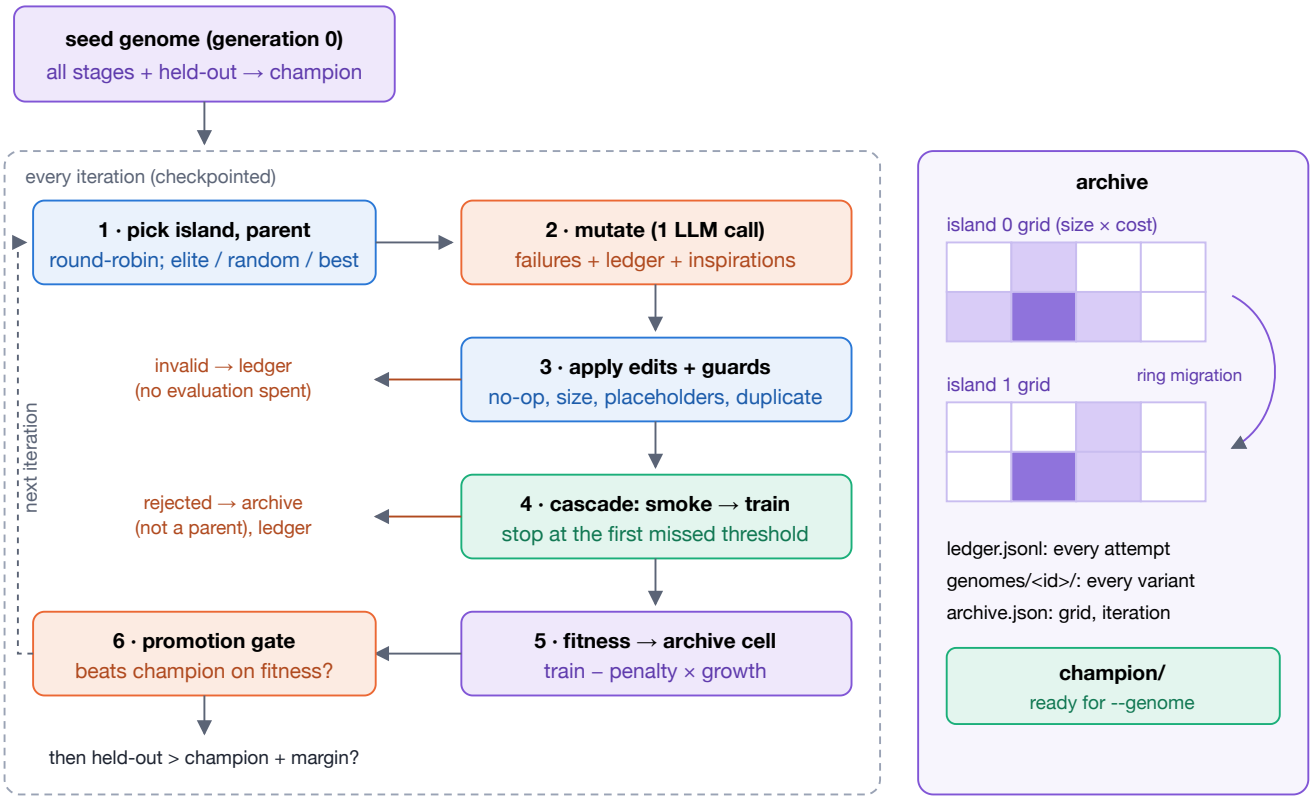


Figure 3. The outer loop. Cheap rejections happen first (guards cost nothing, smoke costs three tasks). The held-out split is spent only on a candidate that would otherwise become champion.

4.1 Seeding

The seed is evaluated on *every* stage regardless of thresholds, plus the held-out split. It is the yardstick. Its size and per-task cost become the archive's reference point for binning. It is placed in every island so none starts empty, and it becomes the first champion. Its fitness is its raw train score (it has no growth to penalize).

4.2 Choosing a parent

Islands are visited round-robin (`iteration mod islands`). Within the island, `sample_parent` draws a random number:

- **50%: exploit.** A random elite, meaning the best occupant of one of the island's grid cells.
- **30%: explore.** Any pooled candidate on the island, elite or not.
- **20%: global best.** The fittest pooled candidate in the whole archive.

The random generator is reseeded with `"<rng_seed>:<iteration>"` each run, so a resumed run does not replay the choices from iteration 0, and two runs with the same seed and evaluator make the same

selections.

4.3 The mutation prompt

One LLM call per iteration (default `claude-opus-5`). The system prompt explains the five artifacts and four rules: diagnose the failure that *recurs* and make 1 to 3 focused edits; **generalize** and never name a specific repository, issue, file, test or answer; **shorter is better**, prefer rewriting or deleting to appending; never repeat an attempt the ledger shows was rejected. The user message carries four things:

Section	Content and caps	Pattern
Parent genome	Every file in full, with its stage scores and fitness	What to change
Failure reports	Up to 6 failures: task id, one-line summary, up to 1,500 chars of detail (failing tests, condensed transcript)	Trace-mined diagnosis
Attempt ledger	Last 20 attempts: summary, outcome, fitness delta, rejection reason	Failure wiki: don't retry dead ends
Inspirations	Two strong, different elites; only the files that differ from the parent, up to 3,000 chars	Recombination of prior art

4.4 The edit format

The mutator answers with a summary line and one or more blocks:

```
<summary>Require a failing reproduction before any fix</summary>
<<<<<<< SEARCH system_prompt.md
- Keep replies brief.
=======
- Keep replies brief.
- Before editing, write a test from the issue text and watch it fail.
>>>>>>> REPLACE
```

Diffs, not whole-file rewrites, because they keep the parent's text that already works and keep proposals small. `apply_edits` is strict where it matters and lenient where models are sloppy:

- An **empty SEARCH** writes the whole file (how new skills and lessons are created).
- An **exact unique match** is replaced. A SEARCH that matches more than once is rejected, because guessing which one was meant is worse than failing.
- No exact match falls back to a **whitespace-tolerant** line match (each line stripped), which must also be unique.
- Anything else (text not found, a path outside the whitelist, editing a file that does not exist) raises `EditError`. The iteration is logged as *invalid* with the reason, which the next mutator sees.

4.5 Guards: free rejections

Before any evaluation is paid for, the child must pass four model-free checks: it is not a **no-op** (fingerprint unchanged); it is under the **size cap** (60,000 chars); the system prompt has not **lost a placeholder** it had; and it is not a **duplicate** of any candidate already in the archive (fingerprint lookup). On a real benchmark each guard hit saves about $10 to $20.

4.6 The cascade

Stages run in order and each has a minimum score, e.g. `--stages smoke:0.67,train`. A child that drops below a threshold stops there and is filed as *rejected*. It stays in the archive and the ledger, but it can never be a parent. On SWE-bench, the smoke stage is three easy tasks the default harness solves. An edit that breaks the agent outright costs three agent runs to discover, not thirteen.

4.7 Fitness

> $\texttt{fitness} = \texttt{score}_{\texttt{last stage}} - \texttt{size_penalty} \times \max(0, \texttt{size} - \texttt{seed_size}) / 1000$
>
> With the default penalty of 0.01, growing the genome by 10,000 characters costs 0.1, which on a 10-task stage is exactly one task. Shrinking is never penalized and never rewarded directly. It wins ties instead: when two candidates have equal fitness, the archive prefers the smaller one. This is the regularized objective. Without it, the easiest route to a higher train score is appending ever more specific advice.

4.8 The archive: MAP-Elites with islands

Greedy hill-climbing on a noisy benchmark locks onto the first lucky variant. The archive keeps an open-ended population instead (the Darwin Gödel Machine argument) and organizes it the MAP-Elites way. Each pooled candidate is binned along two **behavior features** relative to the seed: **genome size** and **per-task run cost**. There are 4 bins per feature spanning 0 to 2× the seed's value, so the seed itself sits mid-range. Each cell of each island's grid keeps only its best occupant. So a lean, cheap variant and a verbose, thorough one survive side by side even when the lean one scores slightly lower, and a later mutation can build on the lean one.

Islands are independent sub-populations searched in turn (default 2). Every 5 iterations, **ring migration** copies each island's best into the next island's grid. A good idea spreads, but no single lineage takes over the archive early.

4.9 The promotion gate

Every pooled child gets a cell, but only one candidate is **champion**, and that is what ships. Promotion is a two-step check:

1. The child's train fitness must strictly beat the champion's. If not, stop: no held-out tasks are spent.
2. Run the held-out stage. The child is promoted only if `holdout > champion.holdout + holdout_margin`.

Held-out failures are never stored on the candidate or shown to the mutator; only the score is kept. That is what makes the split held-out. When a child is promoted, its genome is copied to `<run>/champion/`.

4.10 The ledger and resumability

Every iteration appends one JSON line to `ledger.jsonl`. Each line records the iteration, parent, island and summary, plus an outcome of `seed`, `champion`, `pooled`, `rejected` or `invalid`, with the score, fitness, delta against the parent, held-out score and reason. `archive.json` is written atomically (temp file, then rename) after every iteration. Killing the process at any point and rerunning the same command continues from the last completed iteration, without re-evaluating the seed.

5 · The evaluator contract

The loop never knows what a benchmark is. An evaluator is any object with one method:

```
class Evaluator(Protocol):
    async def evaluate(self, genome: Genome, stage: str) -> EvalResult: ...

@dataclass
class EvalResult:
    score: float                      # in [0, 1]
    metrics: dict[str, float]         # "cost_usd" (per task) feeds the archive grid
    failures: list[Failure]           # (task, summary, detail): what the mutator reads next
    cost_usd: float                   # total spend for this evaluation
```

It is loaded from `--evaluator module:factory` or `--evaluator path/to/file.py:factory` with `--arg key=value` keyword arguments. So benchmark code (Docker, swebench, task files) stays out of the open-source core entirely. **Failures are the most valuable part of the result**: a score says *that* a genome is worse; a failure report says *why*, and the why is what the next mutation acts on.

The tests use a scripted fake evaluator whose score depends on what the genome says. It rewards "run the tests", breaks on "BROKEN" at the smoke stage, and rewards "OVERFIT" on train but punishes it on held-out. The full loop test asserts the exact ledger `seed → champion → invalid → rejected → pooled`: the overfit child improves train fitness but is held back by the held-out gate.

6 · SWE-bench as the evaluator

`bench/evolve_eval.py` implements the contract on SWE-bench Verified: real GitHub issues, one Docker container per task, judged by the repository's hidden tests (`FAIL_TO_PASS` must pass, `PASS_TO_PASS` must keep passing).

6.1 Frozen splits

Stage	File	What it is	Role
smoke	`tasks-smoke3.json`	3 easy tasks the default harness resolves	Did the edit break it?
train	`tasks-hard10.json`	10 from the 1 to 4 h and >4 h tail, stratified by repo. Default harness: 7/10	Fitness, and the failures the mutator reads
holdout	`tasks-holdout10.json`	10 more from the same tail, disjoint from every other set, one per repo before any repeats (`make_split.py --seed 7`)	Promotion gate; score only

The held-out split is drawn from the same difficulty bands as train, so "generalizes" means "to unseen tasks of the same kind," not "to easier ones." The easy ten are deliberately *not* a training stage: the default harness already solves 10/10, so they carry no gradient, only a breakage signal. `make_split.py` refuses to overwrite an existing split. A split that changes breaks comparability across every generation scored on it.

6.2 One evaluation, end to end

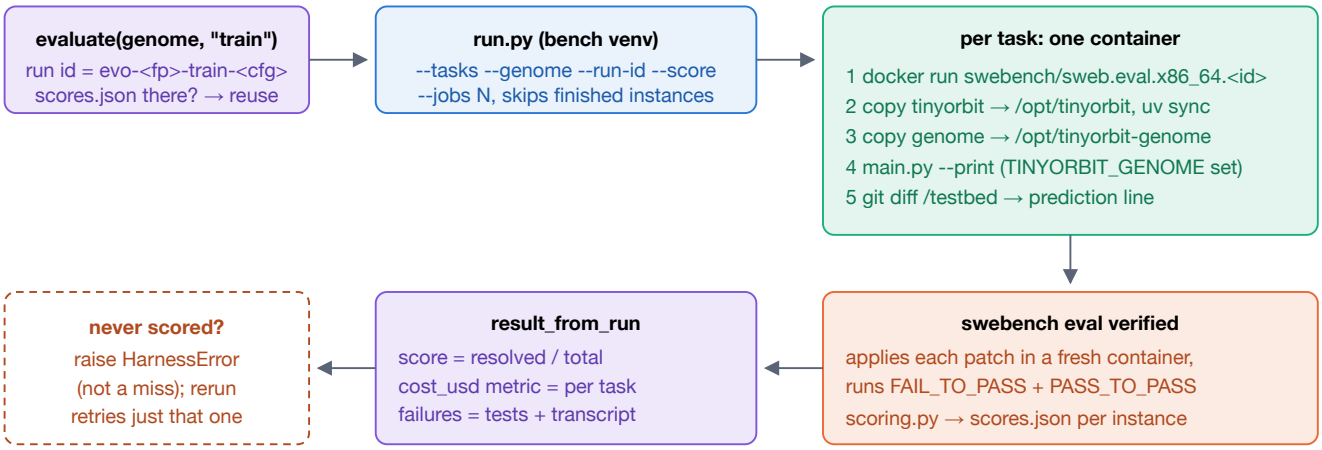


Figure 4. One stage evaluation. The genome is copied outside `/testbed` , so it can never leak into the model's patch.

6.3 Memoisation and resume, at three levels

- **Genome × stage.** The run id is content-addressed: `evo-<genome fingerprint>-<stage>-<settings hash>` , where the settings hash covers model, effort, turn cap and timeout. If two lineages arrive at the same text, or the loop is resumed after a crash, a finished stage is read from disk instead of being paid for again. Different settings never share a cache entry.
- **Instance.** `run.py` skips any instance already present in that run's `predictions.jsonl` . A stage interrupted after 6 of 10 tasks resumes at task 7.
- **Iteration.** The archive itself checkpoints after every iteration (§4.10).

6.4 Infrastructure noise is not a verdict

If a container fails to start, uv cannot install, or swebench errors on an instance, that instance has no prediction or no verdict. Counting it as unresolved would charge the genome for Docker's problem and, on 10 tasks, move its score by 0.1. So `result_from_run` raises `HarnessError` instead. The loop stops, nothing is recorded for that child, and rerunning the same command retries only the missing instance. An agent that times out or produces an empty patch is different: that is a genuine miss and is scored as one.

6.5 What a failure report looks like

For each unresolved instance the mutator receives a summary built from the swebench report and the run's `results.csv`, plus up to 6,000 chars of condensed transcript:

```
task:    astropy__astropy-13398
summary: agent ended with 'completed' but the hidden tests disagree;
         2 target test(s) still fail: test_altaz_with_refraction, ...;
         1 previously passing test(s) broke: test_icrs_gcrs_roundtrip
detail:  USER: <issue> ... ASSISTANT: ... [41 steps omitted] ... RESULT: 3 passed ...
         ASSISTANT: The fix is complete and my new tests pass.
```

Patterns this exposes are exactly the kind the text layer can fix. The default harness's three hard-ten misses all share one: the agent declared success after passing tests it wrote itself, rather than first deriving a failing reproduction from the issue text.

7 · End to end: a run, step by step

7.1 Setup (once)

```
uv sync --group dev                                  # tinyorbit + anthropic dialect
cd bench && uv venv .venv --python 3.11 \
  && uv pip install --python .venv/bin/python swebench  # the scorer, in its own venv
export ANTHROPIC_API_KEY=...                         # agent runs and the mutator
.venv/bin/python run.py --smoke                      # one container, no key: plumbing OK?
```

7.2 Baseline, then evolve

```
# (optional) measure the default harness on held-out, standalone
bench/.venv/bin/python bench/run.py --tasks bench/tasks-holdout10.json \
    --run-id holdout-base --jobs 2 --score

# generation 0 = today's prompt and tool docs, as an editable directory
uv run python -m tinyorbit.evolve seed bench/logs/g0

# the search
uv run python -m tinyorbit.evolve run --out bench/logs/evo-1 --seed bench/logs/g0 \
    --evaluator bench/evolve_eval.py:make --arg jobs=2 \
    --stages smoke:0.67,train --holdout holdout --holdout-margin 0.1 \
    --iterations 10 --islands 2 --rng-seed 0

uv run python -m tinyorbit.evolve show --out bench/logs/evo-1     # leaderboard
open bench/logs/evo-1/report.html                                 # the run, explained
diff -ru bench/logs/g0 bench/logs/evo-1/champion                  # what actually changed
```

7.3 What one iteration does, concretely

1. Iteration 3 → island 1. The draw lands in "exploit": parent `g0002`, an elite.
2. The mutator reads `g0002`'s files, its 3 train failures, the last 20 ledger lines (including "*g0001: added a verify skill, rejected: below train threshold*") and the two other best elites. It proposes one edit to `system_prompt.md` and one new `skills/reproduce/SKILL.md`.
3. Edits apply. Guards pass. The child becomes `g0004` with fingerprint `3f9c…`.
4. Smoke: `run.py` runs 3 containers with the child genome, then swebench scores them: 3/3 ≥ 0.67, continue. Train: 10 containers, 8/10.
5. Fitness = 0.8 − 0.01 × 1.2 (it grew 1,200 chars) = 0.788. Per-task cost $0.95 → cell `1|2|1`. It beats that cell's incumbent and takes the cell.
6. 0.788 > champion's 0.70 → spend the held-out stage: 7/10 against the champion's 6/10 with margin 0.1. 0.7 > 0.6 + 0.1 is false, so no promotion. It stays pooled, a parent candidate for later. Ledger: *pooled, delta +0.088*.
7. `archive.json` is written. Iteration 5 will trigger ring migration.

Every one of those steps is also an event in `events.jsonl`: `iteration_start` (island, parent, pick mode, inspirations), `mutation` (summary, edits, cost, paths to the prompt, answer and diff), `stage_start` / `stage_end` (score, gate, minutes, cost, per-task verdicts, failures, run directory), `candidate`

(fitness, size penalty, grid cell and who held it), `promotion` (yes or no, and why), `migration` and `iteration_end`. `report.py` replays that log into `report.html`, written for a reader who was not watching.

7.4 What lands on disk

```
bench/logs/evo-1/            # the search
  archive.json               # grid,
candidates, champion
  ledger.jsonl               # one line per
attempt
  events.jsonl               # every step,
as it happens
  iterations/NNNN/           # mutator
prompt, answer, diff
  report.html                # rewritten
each iteration
  genomes/g0000 … gNNNN/     # every
variant
  champion/                  # ship this
```

```
bench/logs/evo-<fp>-train-<cfg>/   # one
stage run
  run.json  results.csv  predictions.jsonl
  transcripts/  patches/
  tinyorbit.<run-id>.json
logs/run_evaluation/…
  scores.json               # the reduced
verdicts
bench/logs/evo-genomes/<fp>/      # genome
as run
```

All of `bench/logs/` is untracked: run records stay local. A champion worth keeping is copied out and committed deliberately, along with its scores, as a write-up in `reports/`.

7.5 Using the result

```
uv run python main.py --genome bench/logs/evo-1/champion "fix the failing test"
TINYORBIT_GENOME=bench/logs/evo-1/champion uv run python main.py --print "..."
bench/.venv/bin/python bench/run.py --tasks ... --genome bench/logs/evo-1/champion --score
```

And the inner loop can keep refining a champion between evolution runs, from verified outcomes:

```
uv run python -m tinyorbit.evolve reflect --genome bench/logs/evo-1/champion \
    --transcript bench/logs/<run>/transcripts/<id>.messages.json \
    --outcome "failed" --feedback "FAIL_TO_PASS still failing: test_x"
```

8 · Design choices and where they come from

Choice	Why	Pattern
Evolve text only; model frozen	Narrowest surface that fixes real failures; every variant inspectable and non-executable	Skill/harness layer of the self-evolving stack
Held-out promotion gate	Train improvements that do not transfer are rejected; held-out failures never reach the mutator	Validation-gated skill optimization
Ledger of rejected attempts	The proposer does not rediscover dead ends	Failure wiki
Size penalty; smaller wins ties	Taxes the cheapest overfit (append more hints)	Regularized self-improvement
Archive, not a single best	Noisy scores make greedy search lock onto luck	Darwin Gödel Machine open-endedness
MAP-Elites over size × cost	Keeps lean and thorough variants alive side by side	Quality-diversity search
Islands + ring migration	Parallel lineages; ideas spread without takeover	Island-model evolution
SEARCH/REPLACE diffs, fuzzy fallback	Small proposals, keep what works; tolerate whitespace slips, refuse ambiguity	Diff-based LLM mutation
Staged cascade	Spend 3 tasks to reject what 13 would	Cascade evaluation
Seed = shipped harness, tested	Every delta is measured against a baseline that existed	Behavior-preserving refactor
Content-addressed runs	Same genome is never paid for twice; crash-safe	Memoised evaluation
Harness errors raise	Docker noise is not a verdict on the genome	Fail loud, retry narrow
Evaluator by `file.py:factory`	Benchmarks stay out of the dependency-free core	Plugin seam
Verified outcome into reflection	The agent's own "done" is the least reliable signal	Outcome-grounded reflection

9 · Cost, noise and honest limits

9.1 Budget

From the measured baselines: a hard-tail task costs about **$1.02** and **5.3 min** of agent time, and an easy task about **$0.34** and 2.5 min (claude-opus-5, default effort, 60-turn cap, x86 images under emulation on Apple Silicon).

Event	Agent runs	≈ Cost	≈ Wall time at --jobs 2
Seed (smoke + train + held-out)	23	$21	1 h + scoring
Iteration rejected at smoke	3	$1	5 min
Iteration that reaches train	13	$11	30 to 35 min
Iteration that reaches held-out	23	$21	1 h
5 iterations, typical mix	~80	**≈ $75** ($26 to $130)	3 to 5 h
10 iterations, typical mix	~140	**≈ $135** ($31 to $235)	6 to 10 h

Plus about one mutator call per iteration (cents). Guard rejections and cache hits cost nothing.

9.2 Noise

> - **Ten tasks is a coarse instrument.** One flip moves the score by 0.1, and the same genome can flip a task between runs (the model samples). A 7/10 carries a 95% interval of roughly 35% to 93%. Use `--holdout-margin 0.1` (strictly more than one task) and re-run a champion on held-out before believing it.
> - **Selection on noise.** Taking the best of many noisy evaluations inflates the winner's score (winner's curse). The held-out gate fights this; it does not eliminate it. A final re-evaluation on a fresh split is the honest report.
> - **Contamination.** Frontier models reproduce some Verified gold patches verbatim. "Resolved" does not always mean "reasoned," and harness text cannot change that.

9.3 Known limits

- **Train-set leakage by design.** The mutator reads train failures, including hidden test names. That is the training signal. The rule against naming specific repos or tests is a prompt instruction, not an enforcement. The held-out gate is the actual protection.
- **Live reflection is self-reported.** In `--reflect` mode the outcome is only the agent's exit reason. Use offline reflection with verified outcomes for anything you intend to keep.
- **Fitness reads the last stage.** A child rejected at smoke is compared on its smoke score. That is fine for rejection, and rejected children never become parents, but don't read their fitness as a train estimate.
- **Not yet validated at scale.** The loop, the guards, the archive and the evaluator's parsing are covered by network-free tests. The scoring path has been checked against real Docker and swebench 5.0.2 with gold patches (2 resolved and 1 empty patch, as expected). The images are amd64, so on Apple Silicon `run.py` pre-pulls them before scoring. A genuinely unresolved instance has so far been exercised only by the test fixture, and no full evolution run against the live benchmark has been completed yet.

---

10 · What comes next

1. **First live run**: the seed plus about 5 iterations on the splits above, with the champion re-run on held-out twice. Report it in `reports/` with the full ledger.

2. **Bigger splits**: a 30-task train and 30-task held-out cut the per-flip step to 0.033, at triple the cost. The machinery does not change, only the task files.
3. **Repeated evaluation**: score a would-be champion k times and promote on the mean, to cut variance where it matters most.
4. **Outcome-grounded reflection in the loop**: after each train evaluation, reflect on every failure with its swebench verdict. The child then carries lessons from its own verified misses.
5. **Next layers**: harness code (tool implementations, loop policy) in a sandbox; then curricula, meaning tasks chosen where the archive disagrees most.

A · CLI and file reference

python -m tinyorbit.evolve

Command	Flags
`seed OUT`	Write the built-in harness text as an editable genome (refuses a non-empty dir)
`run`	`--out DIR` (resumes if present) · `--seed DIR` · `--evaluator mod:factory|file.py:factory` · `--arg k=v` · `--iterations 10` · `--stages train` (e.g. `smoke:0.67,train`) · `--holdout NAME` · `--holdout-margin 0.0` · `--size-penalty 0.01` · `--islands 2` · `--rng-seed 0` · `--model claude-opus-5` · `--provider` · `--effort`
`reflect`	`--transcript FILE` · `--genome DIR` · `--outcome` · `--feedback` · `--source` · model flags
`show`	`--out DIR`: candidates sorted by fitness, champion starred
`report`	`--out DIR` · `--html FILE`: replay `events.jsonl` into a self-contained HTML report (story, score chart, family tree, island grids, every attempt)

main.py and bench/run.py

Flag	Effect
`--genome DIR` / `TINYORBIT_GENOME`	Layer a genome over the defaults (both main.py and run.py)
`--reflect` / `TINYORBIT_REFLECT=1`	Print mode: reflect into the genome's lessons.md after the run
`run.py --run-id ID`	Output to `bench/logs/ID/`; reuse to resume
`run.py --jobs N`	N tasks in parallel (and N scoring workers)
`run.py --score`	Run swebench eval, write `scores.json`
`evolve_eval.py:make` args	`jobs` · `model` · `effort` · `max_turns` · `timeout_min` · `python`

Source map

File	Contents
`evolve/genome.py`	Genome, whitelist, fingerprint, seed export, overlay loading, tool-description patching
`evolve/reflect.py`	Lessons format, merge/cap, transcript condensing, the reflection call
`evolve/edits.py`	SEARCH/REPLACE parsing and strict-then-fuzzy application
`evolve/mutate.py`	Mutation system prompt, context assembly, proposal parsing, `LLMMutator`
`evolve/evaluate.py`	`Evaluator` protocol, `EvalResult`, `Stage`, cascade, evaluator loading
`evolve/archive.py`	Candidates, MAP-Elites grid, islands, parent sampling, migration, persistence
`evolve/loop.py`	The driver: seeding, guards, fitness, promotion, ledger, checkpoints
`evolve/events.py`	Append-only `events.jsonl`, per-iteration artifacts, genome diffs
`evolve/report.py`	Replays a run directory into a self-contained HTML report
`evolve/llm.py`	`Complete` seam: any provider as `async (system, user) → str`, cost tracked
`evolve/__main__.py`	CLI
`bench/evolve_eval.py`	SWE-bench evaluator: stages to splits, memoised runs, failure reports
`bench/scoring.py`	swebench output to `scores.json` (stdlib only)
`bench/make_split.py`	Reproducible, repo-stratified, disjoint split builder
`tests/test_evolve.py`, `tests/test_bench_eval.py`	31 network-free tests: overlay identity, edits, lessons, loop outcomes, resume, archive, scoring, memoisation

tinyorbit is MIT-licensed. This document describes branch `feat/self-evolve` at commit 04c1d15 and later.